T03

Danh sách thành viên:

23120027 - Nguyễn Hải Đăng

23120031 - Nguyễn Phú Dinh

23120034 - Ứng Thị Thuỳ Dương

2026 AI-First · CS423/CSC15003

Mobile Automation Testing với Appium 2 và Applitools Eyes



DEMO: Appium 2 + UiAutomator2 (truyền thống) · Applitools Eyes (AI visual)

Video demo: https://youtu.be/N3cN5d2LSG4

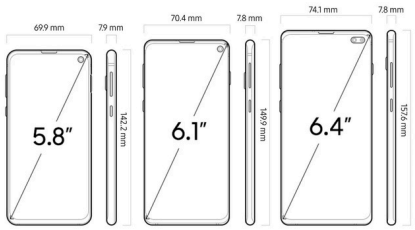
Video theo phần (cài đặt · Record · Data-driven · Eyes): `Demo_Video_Links.md` **· Hướng dẫn chạy lại:** `User_Guide.md`

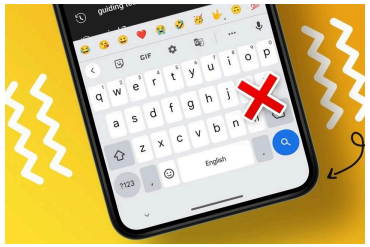
Tại sao kiểm thử mobile khác web và app thường?

Cùng mục tiêu tìm lỗi trước người dùng, nhưng trên mobile **môi trường là một phần của bài test**.

Đặc trưng	Mobile	Web / desktop thường
Phân mảnh máy + OS + DPI / OEM	Rất mạnh	Ít hơn (vài browser / vài OS)
Tương tác	Touch, swipe, long-press, bàn phím ảo che UI	Click, hover, bàn phím vật lý
Vòng đời app	Background, bị OS kill, xoay màn, mở lại giữa chừng	Tab / cửa sổ thường sống lâu
Gián đoạn	Gọi, noti, chuyển app — phải xét phục hồi	Hiếm là hạng mục riêng
Mạng	Wi-Fi ↔ 4G, mất sóng, offline, sync lại	Thường giả định có mạng
Quyền, cảm biến, pin	Runtime permission, GPS/camera, pin	Hạn chế hơn (sandbox trình duyệt)
Phát hành	Store; máy giữ bản cũ lâu	Deploy server / installer

- Cùng locator đúng vẫn có thể fail khi bàn phím che nút, cold start chưa lên foreground, hoặc baseline Eyes gắn nhầm AVD.
- Buổi demo đi sâu **chức năng** và **visual**; các hạng mục còn lại nằm ngoài phạm vi chạy thật.





Kiểm thử mobile gồm những lớp nào?

Phạm vi thường gặp

Khía cạnh	Câu hỏi ngắn
Chức năng	Luồng nghiệp vụ đúng chưa?
Giao diện / visual	Còn giống thiết kế không?
Tương thích	Máy / OS / độ phân giải?
Hiệu năng, pin, mạng	Chậm, tốn pin, mạng yếu?
Gián đoạn	Gọi, noti, chuyển nền?
Cài đặt / quyền	Cài mới, cập nhật, cấp quyền?
Usability / a11y, bảo mật	Dễ dùng? An toàn?

Hai lớp trong demo T03

	Functional	Visual
Hỏi	App **làm** đúng không?	App **trông** đúng không?
Chuẩn so	Giá trị kỳ vọng trong code	Ảnh baseline đã duyệt
Công cụ	Appium	Applitools Eyes

- Functional pass chưa đủ để kết luận giao diện đúng
- Visual pass chưa đủ để kết luận dữ liệu đúng
- Hai lớp bổ sung nhau trên cùng luồng EShop

Khảo sát công cụ — truyền thống và AI-augmented

Truyền thống

Appium 2 — chuẩn WebDriver, đứng ngoài app, Android + iOS, không cần mã nguồn. ✅ **chọn làm lớp functional**

Espresso — chạy trong app, nhanh, chỉ Android, viết Kotlin. ✗

Detox — cho React Native, tự đồng bộ animation/network, cần build riêng. **Đã chạy thử cùng kịch bản: 3/3 lượt pass.** Hợp khi đội RN chỉ cần CI nhanh.

AI-augmented

Applitools Eyes — so ảnh với baseline bằng Visual AI, có ignore/layout region và dashboard duyệt. ✅ **chọn làm lớp visual**

Maestro + AI assertion — flow YAML, `assertWithAI` bằng LLM, không có baseline, không lịch sử duyệt. Dự phòng.

Vì sao ghép Appium + Eyes

Eyes gắn vào **cùng một session** Appium: assert chức năng và checkpoint visual chạy trên cùng luồng, cùng APK người dùng cài.

Quy trình tự động hoá và ba khối nền tảng

Một vòng với EShop

1. Chuẩn bị môi trường (SDK, AVD, APK, backend)
2. Làm app kiểm thử được (`testID` , seed, reset)
3. Thiết kế ca + bảng dữ liệu + oracle
4. Viết script (Page Object, wait, assert, Eyes)
5. Chạy lặp → đọc log → phân biệt lỗi test / môi trường / app
6. Triage visual → Accept / Reject baseline
7. Bảo trì khi app đổi (locator, data, baseline)

Ba khối trong demo

1 Record & Playback

Inspector khám phá cây UI và ghi thao tác. Code Recorder là bản nháp.

2 Data-driven

Cùng một kịch bản, nhiều bộ dữ liệu trong `data/cases.js` .

3 Checkpoints

Assert nghiệp vụ (Appium) và checkpoint giao diện (Eyes).

Cấu trúc bộ test Appium

Bộ test nằm ở `src/appium/`. Code Recorder từ Inspector giữ trong `docs/evidence/` để đối chiếu với Page Object.

```
src/appium/
  config/   session, locators, eyes, checkpoints
  pages/    Page Object theo màn hình
  data/     cases.js — bảng dữ liệu
  tests/    login-search, cart-total
  examples/ search-example.js — file mẫu gom đủ phần
docs/evidence/
  A2_inspector_recorder_….js — code Recorder
```

Recorder so với Page Object

Recorder	Page Object
`$$(...)[0].click()`	`waitForDisplayed` + `click`
Không assertion	Assert text / số / trạng thái
Locator rải trong bước ghi	Locator gom theo màn hình

Diff hai file cho thấy Recorder thiếu wait và assert — đủ để khám phá locator, chưa đủ để chạy ổn định trên CI.

`docs/evidence/A2_inspector_recorder_2026-10-23.js` · `pages/login.page.js`

Nhìn toàn bộ một test — `search-example.js`

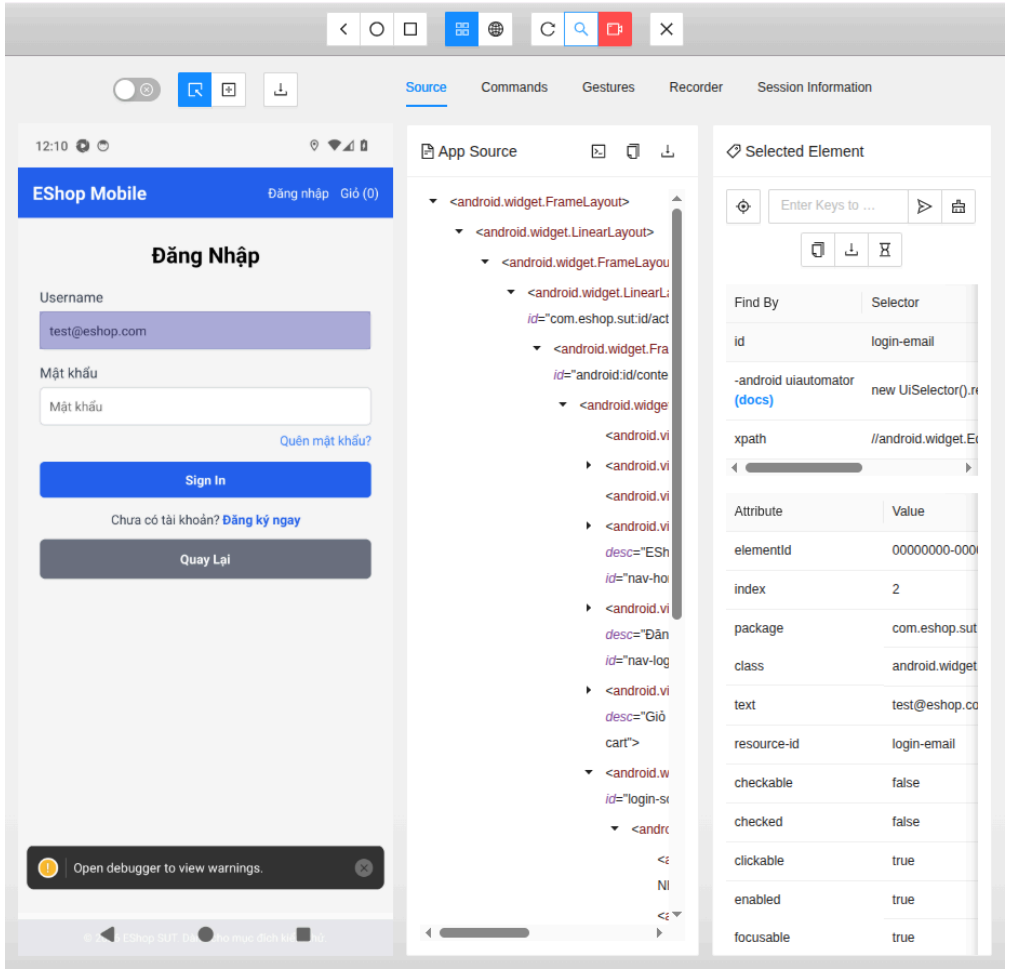
File mẫu gom đủ các phần trong một chỗ. Bộ test chính thức tách cùng cấu trúc ra nhiều file.

1 · Session capabilities, mở phiên → **2 · Locator** `resource-id` từ testID → **3 · Page Object** gom thao tác màn Home → **4 · Data** bảng SEARCH_CASES → **5 · Assert** action → wait → assert → **6 · Eyes** tuỳ chọn, sau assert

```
const SEARCH_CASES = [
  { keyword: "iPhone", count: 1, names: { 1: "iPhone 15 Pro Max" } },
  { keyword: "Pro M",  count: 2, names: { 1: "…", 3: "MacBook Pro M3" } },
];
for (const row of SEARCH_CASES) {
  await t.test(`tìm "${row.keyword}" → ${row.count}`, async () => {
    await home.search(row.keyword);
    await driver.waitUntil(/* … count … */);
    assert.equal(/* tên sản phẩm */, row.names[…]);
  });
}
```

- Chạy riêng: `npm run example` (không nằm trong `npm test`).
- Bộ thật: `config/`, `pages/`, `data/cases.js`, `tests/`.

① Record & Playback — Appium Inspector



Appium Inspector là gì?

Công cụ hỗ trợ quan sát cấu trúc giao diện, xem thuộc tính phần tử và thử các thao tác trên ứng dụng.

Ba tính năng chính

- **Inspect UI Elements:** xem cây giao diện và thuộc tính phần tử.
- **Find & Test Locators:** tìm phần tử bằng locator và thử Tap, Send Keys.
- **Record Actions:** ghi thao tác và sinh code mẫu WebdriverIO.

`testID` trên React Native xuất hiện thành `resource-id` (ví dụ `login-email`).

=> Inspector dùng để **tìm locator và thử thao tác**. Code Recorder là bản nháp; test nộp giữ Page Object kèm wait và assertion.

Quit Session trước khi chạy npm test

② Data-driven — cùng kịch bản, nhiều bộ dữ liệu

Dữ liệu tách khỏi logic: thêm một trường hợp là thêm một dòng trong `data/cases.js`.

Bảng dữ liệu

ID	Đầu vào	Kỳ vọng
L1	sai mật khẩu	còn Login + `login-error`
L2	đúng tài khoản	“Chào, Test User”
S1	`iPhone`	1 kết quả
S2	`Pro M`	2 kết quả
S3	`Nokia 3310`	0 kết quả
Cart	iPhone × 2	tổng chữ số `60000000`

Vòng lặp trong test

```
const { SEARCH_CASES } = require("../data/cases");

for (const row of SEARCH_CASES) {
  await t.test(
    `${row.id} "${row.keyword}"`,
    async () => {
      await home.searchFor(row.keyword);
      // assert theo row.expectedCount / row.expected
    }
  );
}
```

- Mỗi dòng bảng → một subtest
- Assertion lấy kỳ vọng từ dòng đó
- Mở rộng bộ dữ liệu: thêm object vào mảng

LOGIN_CASES cùng kiểu; thứ tự L1→L2 cố định vì backend khoá tài khoản sau nhiều lần sai.

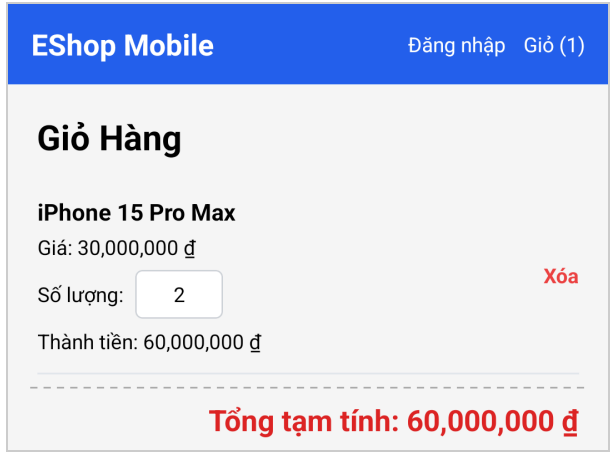
③ Checkpoint chức năng — Locator, Action & Assertion

Kịch bản — Verify Cart Total: thêm *iPhone 15 Pro Max* (30 000 000 ₫) vào giỏ 2 lần → tổng tạm tính phải là **60 000 000 ₫**.

1 · Locator — Tìm nút "Thêm vào giỏ" (`product-add-1`) → **2 · Action** — Bấm 2 lần, mở giỏ hàng → **3 · Wait** — Chờ dòng tổng tiền hiện ra → **4 · Assertion** — Tổng = 60 000 000 ₫? → **Test Result** — PASS / FAIL dựa trên Assertion

```
const byId = (id) => driver.$(`android=new UiSelector().resourceId("${id}")`);
const addBtn = await byId("product-add-1");          // 1 Locator
for (let i = 0; i < 2; i++) {                        // 2 Action: thêm 2 lần
  await addBtn.click();
  await (await byId("android:id/button1")).click();  // đóng hộp thoại "Thành công"
}
await (await byId("nav-cart")).click();
const total = await byId("cart-total");
await total.waitForDisplayed({ timeout: 10_000 });   // 3 Wait: chờ điều kiện
assert.equal((await total.getText()).replace(/\D/g, ""), "60000000");  // 4 Assertion
```

Best practice: Explicit Wait — chờ điều kiện cụ thể thay vì thời gian cố định. Bộ test thật gom các bước vào Page Object (`home.addToCart(1, 2)`).



Màn giỏ hàng EShop lúc test chạy trên emulator (23/10/2026).

Test này pass vì tổng tiền đúng. Nhưng nếu dòng tổng lệch sang trái và nút thanh toán đổi màu, Appium có phát hiện được không? → Applitools Eyes

Đọc kết quả — Pass, Fail và nhiều bộ dữ liệu

`npm test` chạy **3 test**, trong đó Login và Search lặp theo bảng. TAP đếm từng subtest.

Pass

- L1: thấy `login-error`, vẫn ở màn Login - L2: thấy “Chào, Test User” - S1 / S2 / S3: đúng số kết quả (1 / 2 / 0) - Cart: tổng chữ số = `60000000` → `# pass 8` `# fail 0`

Fail

- Sai mật khẩu mà không có thông báo lỗi - Tìm `iPhone` mà số kết quả ≠ 1 - Tổng giỏ ≠ `60000000` → Chỉ subtest đó đỏ; các dòng còn lại vẫn chạy

Chạy theo bảng dữ liệu

Trong session	Mỗi dòng `cases.js` thành một subtest, chạy tuần tự
Trên TAP	Mỗi dòng có dòng `# Subtest …` riêng
Thêm bộ dữ liệu	Thêm object → lần sau tăng số kiểm tra (ví dụ 8 → 9)
Eyes	Mỗi bước quan trọng một checkpoint trong cùng batch

Applitools Eyes — Checkpoint và Baseline

Checkpoint	Ảnh chụp lần chạy hiện tại, tại một bước đã ổn định (sau assert)
Baseline	Ảnh đã Accept — chuẩn để so lần sau
Visual diff	Vùng Eyes tô khi checkpoint lệch baseline
Match level	Strict, Layout…; có thể gắn ignore / layout region

Từ chụp đến duyệt

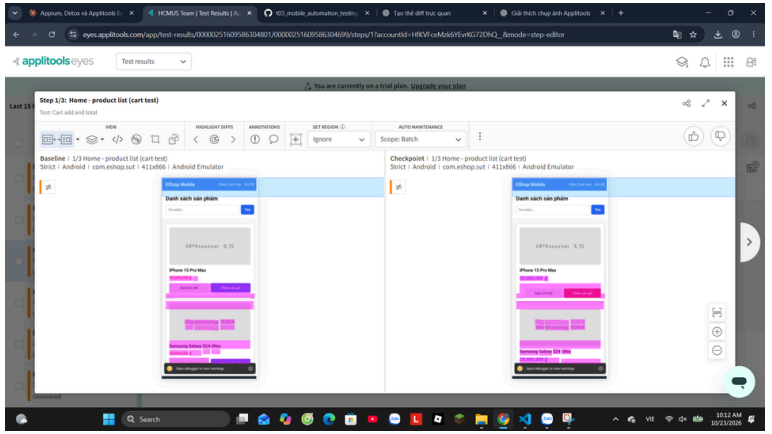
1. Đưa app tới trạng thái đã biết
2. `eyes.check(...)` tạo checkpoint
3. So với baseline cùng tên bước và cùng cấu hình máy
4. Khớp → Passed; lệch → Unresolved
5. Accept → thành baseline mới; Reject → ghi nhận lỗi

Lần đầu chưa có chuẩn: trạng thái New; Accept lần đầu tạo baseline.

Cùng session Appium

```
await openEyes(eyes, driver, "Cart add and total");
await checkpointAs(eyes, "cartFilled",
  "Cart - 1 item with total");
await closeEyes(eyes);
// Eyes: Passed | … | link dashboard
```

- Eyes so ảnh, không đọc giá trị tiền như assertion.
- Baseline gắn một AVD; đổi máy thường cần baseline mới.



Trái: baseline · Phải: checkpoint · vùng tô: diff

Xem kết quả Eyes trên dashboard

Sau khi chạy, terminal in trạng thái và link batch. Chi tiết so ảnh nằm trên **Eyes Cloud**.

Từ terminal tới dashboard

1. Chạy test với `APPLITOOLS_API_KEY`
2. Terminal: `Eyes: Passed | Unresolved | …` + URL
3. Mở Applitools Dashboard
4. Chọn test → từng step là một checkpoint
5. Xem baseline / checkpoint / diff
6. Accept hoặc Reject từng bước

Khi demo / CI

- `eyes.close(false)`: TAP chức năng vẫn có thể `# fail 0` khi có diff visual
- Cần mở dashboard để triage
- Ảnh lưu sẵn: `docs/evidence/B1_B2_applitools-dashboard/`

Ba bản giao diện

Bản	Appium	Eyes
base	8/8 pass	Passed
V1 đổi ảnh SP	8/8 pass	Unresolved 5/10
V2 đổi màu + lệch layout	8/8 pass	Unresolved 6/10

- V1: đổi chủ đích → thường Accept hoặc Layout region
- V2: regression giao diện mà functional không thấy → Reject
- Clip: youtu.be/… (4:06)

Tính năng của hai công cụ

Appium

- Điều khiển APK qua WebDriver: mở app, nhập liệu, nhấn nút.
- Record (Inspector), data-driven và assertion chức năng.
- Đọc text / trạng thái để xác minh nghiệp vụ (đăng nhập, tìm kiếm, tổng giỏ).
- Chạy trên emulator, thiết bị thật hoặc device farm.

Applitools Eyes

- So màn hình hiện tại với baseline bằng Visual AI.
- Hỗ trợ match level và ignore / layout region.
- Dashboard để Accept hoặc Reject từng checkpoint.
- Gắn cùng session Appium sau bước assert chức năng.

Hai lớp bổ sung nhau — Appium không thay Eyes; Eyes không thay assert giá hay quyền

Lưu ý khi dùng: Appium cần wait và locator ổn định; Eyes cần người duyệt baseline và không bắt lỗi không hiện trên ảnh.

Visual Diff Triage Workshop

Q&A

Video demo: https://youtu.be/N3cN5d2LSG4

`Demo_Video_Links.md` · `User_Guide.md`